

Foundations of game design

Relazione seminario - Generazione dei mondi di gioco secondo
tecniche procedurali

Autore: Marco Ferioli - matricola 879277

Data: 8 settembre 2026

Sommario

Uno degli aspetti più importanti nell'ambito di sviluppo dei videogiochi riguarda la generazione procedurale dei mondi di gioco, perché consente al giocatore di condurre un'esperienza differente in ogni partita, garantendo un alto livello di rigiocabilità. La seguente relazione esplora ed implementa alcune tecniche utilizzate nell'ambito della generazione di terreni con tecniche procedurali, come il Perlin noise, il Poisson disk sampling e la wave function collapse. Ciascun algoritmo ha delle funzionalità specifiche; in particolare la prima tecnica consente di modellare il terreno, mentre la seconda e la terza sono dedicate al posizionamento degli assets (alberi, rocce, case) con lo scopo di modellare l'ambiente. La combinazione di tutte queste metodologie consente di produrre una vera e propria pipeline di generazione procedurale; quest'ultima è stata implementata utilizzando la libreria grafica Three.js ed applicando tecniche programmatiche di separazione degli interessi, con l'obiettivo di rendere modulare ciascun aspetto della procedura.

1 Introduzione

La generazione di mondi di gioco che siano differenti in ogni partita rappresenta un aspetto fondamentale di molteplici videogiochi di successo mondiale, come *Minecraft*, *Terraria* o *No man's sky*. L'utilizzo di algoritmi di generazione procedurale consente, in particolare, di creare ambienti differenti per ogni partita, massimizzando le possibilità di giocare nuovamente il videogioco ed evitando di progettare manualmente ogni porzione del mondo che verrà esplorato dal giocatore.

L'obiettivo dell'attività proposta è analizzare, comprendere e confrontare alcune delle tecniche algoritmiche più utilizzate nell'ambito della creazione di mondi procedurali, definendo le modalità di applicazione e i limiti di ciascuna metodologia. In particolare, ciascuna tecnica consente di gestire un aspetto preciso della generazione procedurale; per questa ragione, è possibile affermare che esistono molteplici soluzioni differenti riguardo alla generazione di mondi di gioco in modo casuale. Il compito di chi progetta il videogioco è quello di selezionare la pipeline (e quindi gli algoritmi associati) più adatta all'obiettivo da implementare.

Dato che il progetto proposto ha un carattere principalmente dimostrativo riguardo agli algoritmi di generazione procedurale e non richiede un motore fisico, è stata utilizzata la libreria grafica Three.js con lo scopo di modellare un terreno di gioco di dimensione 64×64 , di distribuire gli assets (alberi, rocce, ecc.) su tale terreno e di effettuare il rendering della scena. Tale libreria ha un funzionamento simile, ma semplificato, rispetto ai classici game engine (come Unity), perché permette di creare scene, aggiungere mesh e camere con cui mettere in atto l'operazione di rendering; inoltre supporta l'utilizzo di shaders personalizzate.

La pipeline implementata si basa sull'applicazione di alcuni algoritmi, come l'improved Perlin noise [5], il Poisson Disk Sampling [1] e il Wave Function Collapse [2] ed è suddivisa in due porzioni fondamentali. Nella prima parte viene effettuata una modifica della struttura del terreno con lo scopo di modellare l'ambiente (pianure, colline, montagne), mentre nella seconda parte vengono utilizzate tecniche di distribuzione degli assets per modellare vari biomi, come le foreste o i villaggi.

1.1 Struttura generale del progetto

Il progetto è organizzato secondo una struttura gerarchica ben definita. In primo luogo è necessario selezionare la definizione del piano che verrà modellato per rappresentare il terreno. Questa operazione avviene scegliendo il numero di vertici che compongono la geometria (Figura 1); maggiore è il numero di vertici, maggiore sarà la risoluzione del terreno. Per semplicità è stato scelto di assegnare a ciascuna dimensione del piano lo stesso numero di vertici N .

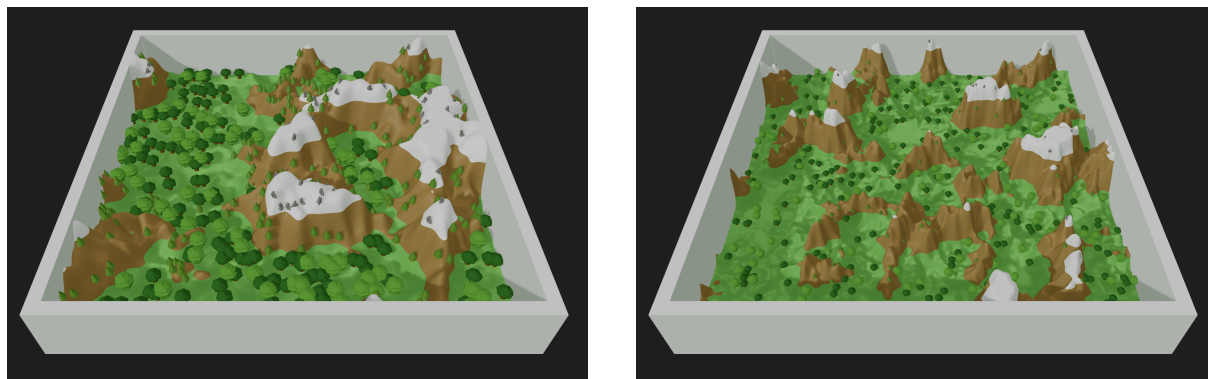


Figura 1: Confronto tra piani con risoluzione (vertici) pari a 256×256 e 512×512

A questo punto viene generata una height map H , ossia una matrice di valori nell'intervallo $[0, 1]$ di dimensione $N \times N$, che, sfruttando un algoritmo, definisce l'altezza H_{ij} a cui deve essere posizionato ciascun vertice del terreno.

Successivamente la matrice H viene utilizzata per costruire una asset map A , ossia una matrice binaria di dimensione $M \times M$ con $M < N$; l'obiettivo di quest'ultima consiste nell'indicare dove devono essere posizionati gli assets sul terreno. La dimensione della matrice A è minore di quella della matrice H , perché ciascun asset deve occupare una porzione di spazio nel piano; qualora fosse deciso di porre $M = N$, gli assets posizionabili possibili sarebbero pari al numero di vertici e ciò risulterebbe in un agglomerato di assets eccessivo. In fase di costruzione, invece, vengono utilizzati dei vincoli per definire quale asset deve essere disegnato a schermo.

L'ultimo passaggio consiste nel creare una matrice V (village map) di dimensione $K \times K$ con $K < M$, la quale contiene la struttura del villaggio; quest'ultimo viene quindi inserito dinamicamente all'interno dell'asset map. Un aspetto particolare riguarda la scelta della posizione del villaggio; infatti è necessario predisporre un algoritmo di ricerca della migliore posizione rispetto alla matrice delle altezze H .

La distinzione tra le varie matrici risulta fondamentale dal punto di vista programmatico, perché consente di separare le responsabilità e gli interessi di ciascuna classe, oltre a consentire l'utilizzo modulare; la loro combinazione, invece, consente di conoscere dove posizionare ciascun asset. Dato che le varie matrici hanno risoluzione differente, sono stati predisposti degli algoritmi con l'obiettivo di effettuare la conversione delle coordinate da una tipologia all'altra; in questo modo è possibile garantire una corrispondenza coerente tra le varie posizioni.

2 Generazione della heightmap

2.1 Approccio con riempimento casuale

La generazione della matrice H rappresenta un passaggio cruciale nella definizione della pipeline di generazione procedurale, perché permette di modellare il piano su cui verranno posizionati i vari assets. La procedura di generazione dovrebbe avere una componente casuale, che garantisce di cambiare il risultato per ogni generazione, ed una componente di omogeneità, che consente di modellare terreni in modo tale che appaiano naturali. Ad esempio, è possibile prendere in considerazione il seguente algoritmo di riempimento, che assegna a ciascuna cella della matrice H un numero casuale nell'intervallo $[0, 1]$.

Algorithm 1: Generazione della matrice H con valori casuali

Input: N , numero di vertici del piano

Output: Height map $H \in \mathbb{R}^{N \times N}$ con $H_{i,j} \in [0, 1]$

```
 $H \leftarrow \text{new Matrix}(N \times N);$   
for  $i \leftarrow 1$  to  $N$  do  
  for  $j \leftarrow 1$  to  $N$  do  
     $H_{i,j} \leftarrow \text{random}(\text{min} = 0, \text{max} = 1);$   
return  $H;$ 
```

Il principale problema di tale procedura riguarda la naturalezza del risultato. Il terreno risulta essere generato casualmente, ma non esiste alcun tipo di vincolo che lega il valore di una cella a quello delle celle circostanti. Di conseguenza, come è possibile notare dall'immagine successiva (Figura 2), il risultato consiste in una mappa non utilizzabile, perché la variazione tra celle vicine risulta essere non controllata.

Al contrario, l'obiettivo ricercato consiste nel generare una matrice H dove la generica cella H_{ij} abbia una variazione contenuta rispetto a tutte le celle che l'affiancano. Per realizzare tale proposito è possibile prendere in considerazione algoritmi di generazione del rumore ed utilizzarli per la creazione della height map. Nel caso preso in considerazione, è stato scelto il Perlin noise [4] ed in particolare è stata implementata la variante "improved" [5].

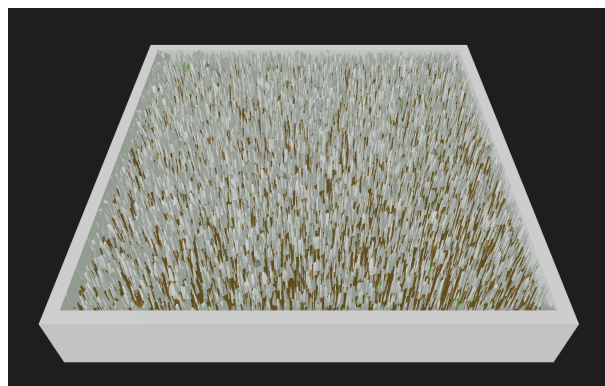


Figura 2: Risultato dell'asset map generata casualmente

2.2 Algoritmo di generazione del Perlin noise

Il Perlin noise [4] è un algoritmo che consente di generare rumore coerente, cioè una variazione pseudocasuale continua e naturale; ciò permette di modellare la correlazione spaziale tipica degli ambienti naturali. L'algoritmo riceve in input un insieme di coordinate $(c_1, \dots, c_N) \in \mathbb{R}^N$, che rappresentano un punto nello spazio di riferimento, e restituisce un valore $v \in \mathbb{R}$ che rappresenta il rumore in quel punto.

Dato che in questo caso l'obiettivo finale dell'algoritmo è popolare una matrice, lo spazio di riferimento è formato da due dimensioni e le coordinate sono espresse tramite una coppia di valori (x, y) . Lo spazio di riferimento viene suddiviso in celle quadrate i cui lati hanno dimensione unitaria tramite l'applicazione di una griglia; in particolare ogni vertice della griglia ha associato in modo deterministico un vettore nella forma $g = (g_x, g_y)$ scelto in modo pseudocasuale (tramite una permutazione P dei primi 256 valori interi) tra un set G di vettori normalizzati in modo tale che rappresentino una direzione.

Dato un punto $p = (x, y)$, questo si troverà all'interno di una specifica cella di vertici $V = [A, B, C, D]$ appartenente alla griglia definita in precedenza. È quindi necessario elaborare, per ciascun vertice della cella, il vettore che collega il vertice al punto in esame; per mettere in atto tale operazione è sufficiente applicare la differenza tra il punto p e ciascun vertice v_i : $d_i = p - v_i$.

A questo punto si procede facendo il prodotto scalare tra ciascuna distanza d_i con il gradiente g_i del vertice associato; in particolare si ha $\eta_i = g_i \cdot d_i = g_x \cdot d_x + g_y \cdot d_y$. Questa operazione serve a misurare l'allineamento del punto rispetto alla direzione indicata dal gradiente e definisce il contributo di ogni singolo vertice al rumore.

Infine, l'ultimo passaggio consiste nell'effettuare un'interpolazione lineare sfruttando i dati appena calcolati. In primo luogo è necessario calcolare la posizione relativa del punto all'interno della cella; tale posizione può essere indicata con $p_f = (x_f, y_f) = (x - \lfloor x \rfloor, y - \lfloor y \rfloor)$. La versione "improved" dell'algoritmo [5] suggerisce di applicare la funzione $fade(t) = 6t^5 - 15t^4 + 10t^3$, perché consente di controllare meglio la continuità. In secondo luogo, viene applicata l'interpolazione $lerp(a, b, t) = a + t(b - a)$ ai prodotti scalari calcolati in precedenza:

$$v = lerp(lerp(\eta_A, \eta_B, fade(x_f)), lerp(\eta_C, \eta_D, fade(x_f)), fade(y_f))$$

Il risultato ottenuto dall'applicazione della funzione rappresenta il valore del Perlin noise nel punto $p = (x, y)$. Di seguito sono indicati tramite pseudocodice gli algoritmi utilizzati.

Algorithm 2: Calcolo del vettore associato a un vertice della cella

Input: $(i, j) \in \mathbb{Z}^2$, permutazione P , set di vettori normalizzati G

Output: $g = (g_x, g_y) \in G$

$h \leftarrow P[(P[i \bmod 256] + j) \bmod 256];$

return $G[h \bmod |G|];$

Algorithm 3: Calcolo dell'Improved Perlin noise in $\mathbb{R}^2$

Input: $(x, y) \in \mathbb{R}^2$, permutazione P , insieme dei vettori G **Output:** v

```
(i, j)  $\leftarrow$  ( $\lfloor x \rfloor$ ,  $\lfloor y \rfloor$ );  
p  $\leftarrow$  (x, y);  
(xf, yf)  $\leftarrow$  (x - i, y - j);  
A  $\leftarrow$  (i, j), B  $\leftarrow$  (i + 1, j), C  $\leftarrow$  (i, j + 1), D  $\leftarrow$  (i + 1, j + 1);  
foreach q  $\in$  {A, B, C, D} do  
    gq  $\leftarrow$  getVector(q, P, G);  
    dq  $\leftarrow$  p - q;  
    ηq  $\leftarrow$  gq · dq;  
tx  $\leftarrow$  fade(xf);  
ty  $\leftarrow$  fade(yf);  
return lerp(lerp(ηA, ηB, tx), lerp(ηC, ηD, tx), ty);
```

2.3 Perlin noise con ottave

Dato che l'algoritmo del Perlin noise si basa su uno spazio continuo, mentre le coordinate della matrice da popolare sono discrete, si rivela necessario stabilire una corrispondenza tra i due sistemi. Le coordinate della matrice devono essere moltiplicate per un fattore di scala k , la cui funzionalità è quella di impedire campionamenti eccessivamente distanti nello spazio del Perlin noise, il cui risultato è simile al popolamento casuale della matrice.

Il Perlin noise rappresenta un algoritmo che da solo non è in grado di produrre sufficiente variabilità locale. Come è possibile notare nella Figura 3, l'utilizzo del Perlin noise produce delle configurazioni molto morbide, che però non presentano le tipiche irregolarità che caratterizzano ad esempio le pianure o le pareti montuose. Per risolvere questo problema è sufficiente effettuare più campionamenti con fattori di scala decrescenti e sommarli tra loro (Figura 3). Ciascun campionamento è detto "ottava" [3] e contribuisce al valore finale tramite un peso detto amplitude; di conseguenza il Perlin noise può essere considerato come il risultato di una somma pesata.

Algorithm 4: Calcolo del Perlin noise mediante ottave

Input: $(x, y) \in \mathbb{R}^2$, numero di ottave O **Output:** $v \in \mathbb{R}$

```
v  $\leftarrow$  0;  
α  $\leftarrow$  1; // ampiezza  
F  $\leftarrow$  1; // frequenza  
for i  $\leftarrow$  1 to O do  
    v  $\leftarrow$  v + α · PerlinNoise(Fx, Fy);  
    F  $\leftarrow$  2F;  
    α  $\leftarrow$   $\frac{\alpha}{2}$ ;  
return v;
```

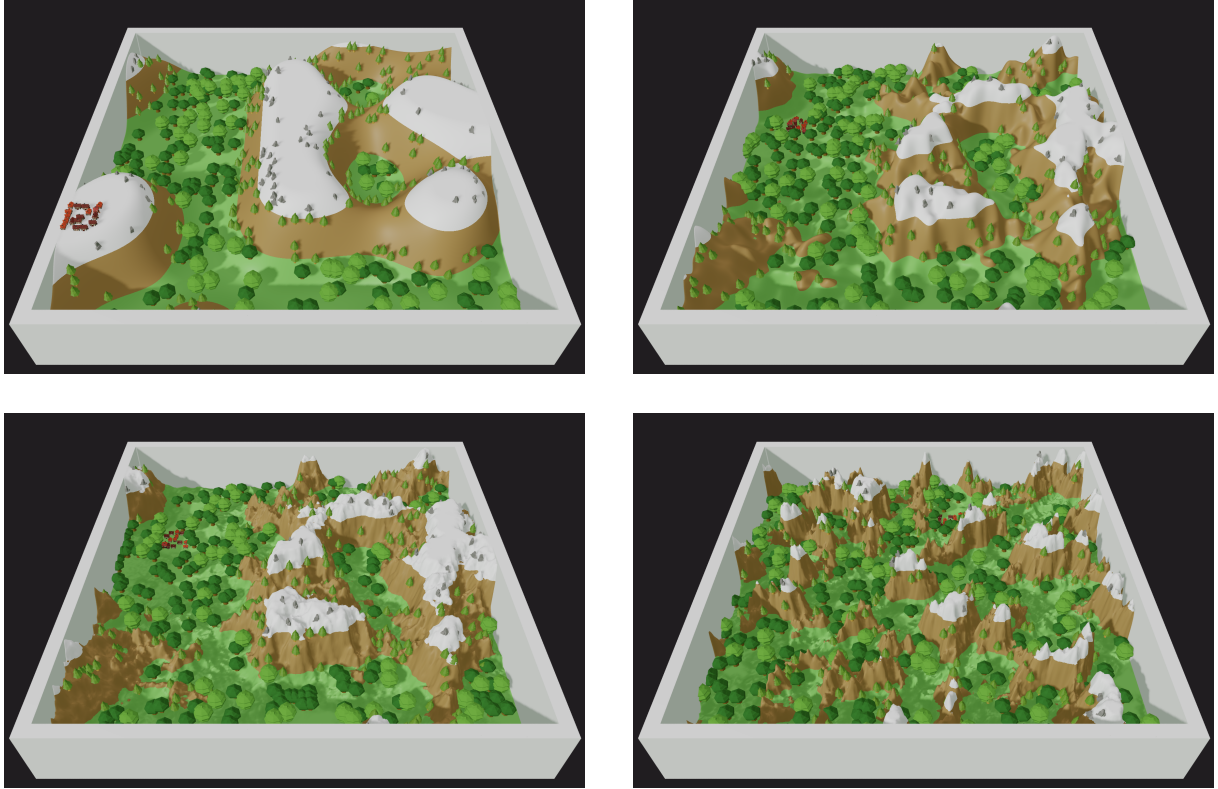


Figura 3: Confronto tra piani calcolati con Perlin noise con differenti ottave (1, 4, 8) e fattore di scala $k = 0.01$. La quarta immagine, invece, contiene un piano generato con fattore di scala $k = 0.03$.

2.4 Ulteriori operazioni messe in atto

La matrice popolata tramite l'applicazione del Perlin noise viene convertita in una texture in scala di grigi e fornita ad una *vertex shader*, la quale si occupa di posizionare i vertici del piano secondo i valori campionati. In questa fase vengono applicate due funzioni: $\text{pow}(\text{value}, \text{power})$ e $\text{smoothstep}(0.2, 0.9, \text{value})$. La prima funzione mette in atto una trasformazione non lineare, mentre la seconda si occupa di portare a 0 i valori più piccoli di 0.2 ed a 1 i valori più grandi di 0.9. La loro combinazione e la scelta del valore *power* consente di gestire il rapporto tra le pianure e le montagne presenti sulla mappa (Figura 4). Infine, un fragment shader consente di colorare l'ambiente sulla base dell'altezza.

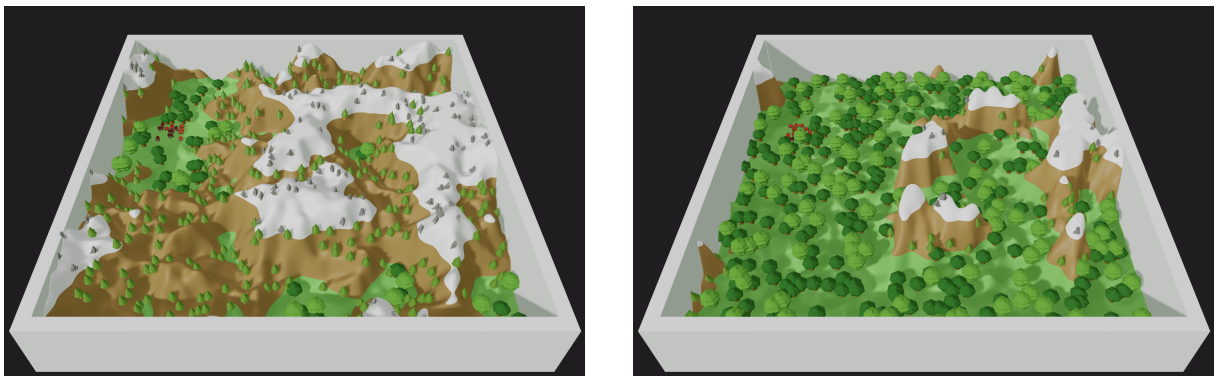


Figura 4: Confronto tra piani le cui height matrices sono state manipolate con $\text{power} = 1$ e $\text{power} = 10$

3 Disposizione degli assets

La matrice di distribuzione degli assets A è costruita in modo molto semplice. La prima operazione da mettere in atto consiste nello scegliere quanto spazio può occupare ciascun asset sul piano; ad esempio, avendo un piano con 256×256 vertici e selezionando una asset resolution γ pari a 4, si avrà come risultato una asset map di dimensione 64×64 . Ciò significa che il piano ha la possibilità di presentare un asset ogni 4×4 vertici.

L'obiettivo della pipeline è popolare la matrice binaria degli assets A in modo tale che se $A_{ij} = 1$, allora nella cella A_{ij} è presente un asset. In una seconda fase, le coordinate della matrice A vengono convertite negli indici associati alla matrice H ; in questo modo è possibile calcolare l'altezza (tramite media) a cui deve essere posizionato ciascun asset. Sfruttando le altezze vengono altresì definiti gli assets che devono essere disegnati a schermo; ad esempio le querce possono trovarsi solo ad altezza compresa nell'intervallo $[0, 0.4]$ (pianura), mentre le rocce devono comparire solo ad altezza $[0.9, 1]$.

L'aspetto critico è rappresentato dalla procedura messa in atto per popolare la matrice. Ad esempio, utilizzando una tecnica di assegnazione casuale, gli assets vengono distribuiti equamente lungo la mappa; il risultato consiste in una struttura comunque apprezzabile, ma che non ha coerenza con gli ambienti che si vogliono rappresentare. Prendendo in considerazione gli alberi, generalmente possono fare parte di foreste, che nel caso dell'algoritmo di distribuzione casuale degli assets non possono essere rappresentate. Per risolvere questo problema è stato implementato l'algoritmo "Poisson disk sampling" [1]. L'idea alla base della procedura messa in atto consiste nel selezionare dei punti casuali equidistanti all'interno della mappa; tali punti fungono da "cluster", perché gli altri assets sono distribuiti attorno ad essi.

L'algoritmo mantiene due insiemi separati: *samples*, che contiene tutti i punti validi generati ed *activeList*, che invece contiene tutti i punti da cui è possibile generare nuovi campioni. La struttura generale è riassunta dai seguenti punti:

1. Si sceglie casualmente un punto e lo si inserisce sia in *samples*, che in *activeList*.
2. Finché *activeList* non risulta vuota si seleziona uno dei suoi elementi e si generano fino a k candidati casuali nell'anello di raggio compreso tra r e $2r$. Per ogni candidato si verifica che appartenga al dominio e che sia distante almeno r da tutti i punti vicini. Nel caso in cui questi vincoli siano rispettati, il punto trovato viene aggiunto ad entrambi gli insiemi *samples* ed *activeList*. Se dopo k tentativi non viene trovato nessun candidato valido, il punto viene rimosso dall'*activeList*.
3. Quando *activeList* diventa vuota, l'insieme *samples* costituisce il risultato finale.

Dato che in una mappa procedurale potrebbero esistere un certo numero massimo di foreste, è stato implementato un ciclo iterativo che effettua la chiamata alla funzione di Poisson Disk Sampling fino a quando la dimensione di *samples* restituita non è pari al numero richiesto di foreste. Ovviamente, al variare delle iterazioni, diminuisce il valore del raggio r con cui è chiamata la funzione; maggiore è il numero di foreste, minore può essere la distanza che intercorre tra esse.

Algorithm 5: Poisson Disk Sampling

Input: distanza minima r , numero di tentativi k

Output: insieme dei punti S

Genera un punto casuale p ;

$S \leftarrow \{p\}$;

$A \leftarrow \{p\}$;

while $A \neq \emptyset$ **do**

 Estrai un punto q da A ;

for $i \leftarrow 1$ **to** k **do**

 Genera un candidato c a distanza $[r, 2r]$ da q ;

if c è *valido* **then**

$S \leftarrow S \cup \{c\}$;

$A \leftarrow A \cup \{c\}$;

if *nessun candidato valido è stato trovato* **then**

$A \leftarrow A \setminus \{q\}$;

return S ;

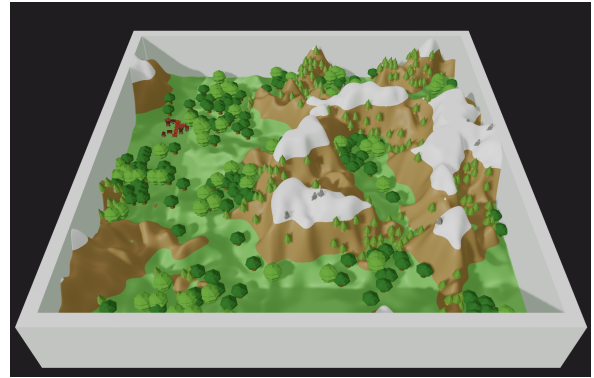
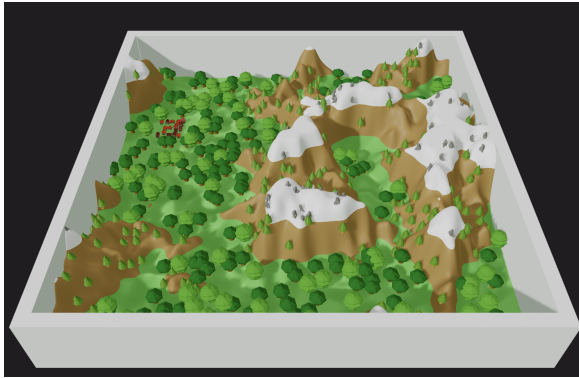


Figura 5: Differenza tra la distribuzione degli assets effettuata in modo casuale ed utilizzando il Poisson disk sampling per generare 10 foreste (l'80% degli assets è assegnato alle foreste ed il restante è distribuito casualmente)

4 Generazione del villaggio

Spesso, all'interno di una mappa, è possibile incontrare strutture come villaggi. L'ultimo passaggio della pipeline implementata consiste nel generare un villaggio tramite tecniche procedurali. Supponendo che il villaggio abbia dimensione $K \times K$ e che l'asset map abbia dimensione $M \times M$ con $K < M$, l'idea alla base della generazione procedurale consiste nel determinare la migliore posizione in cui il villaggio deve essere costruito e la sua struttura. Per mettere in atto la prima operazione viene sfruttato un semplice algoritmo di ricerca che si basa su una finestra scorrevole di dimensione $K \times K$. In particolare viene selezionata la finestra le cui celle hanno varianza minima; ciò significa che il terreno risulta essere stabile (senza pendenze) in una specifica porzione.

Algorithm 6: Ricerca della posizione ottimale per un villaggio

Input: height map H , dimensione del villaggio K , asset resolution r

Output: coordinate (x, y) del villaggio

```
 $var_{min} \leftarrow +\infty;$   
 $(x, y) \leftarrow (-1, -1);$   
for  $i \leftarrow 0$  to  $M - K$  do  
    for  $j \leftarrow 0$  to  $M - K$  do  
        Calcola la varianza delle altezze nella finestra  $K \times K$  con origine  $(ir, jr)$ ;  
        if  $var < var_{min}$  then  
             $var_{min} \leftarrow var;$   
             $(x, y) \leftarrow (i, j);$   
return  $(x, y);$ 
```

4.1 Wave Function Collapse

La procedura di Wave Function Collapse è utilizzata per definire dinamicamente la creazione della struttura del villaggio. L'algoritmo si basa su un insieme di passaggi ben definiti:

1. Definizione della struttura di base ed estrazione delle regole. Il primo passaggio consiste nella definizione di una struttura da cui è necessario estrarre un insieme di regole. Nel caso preso in considerazione, tale struttura è rappresentata da una semplice matrice di valori interi, dove ciascun numero intero è identificativo di uno specifico asset che deve comparire nel villaggio. Ad esempio, nel caso preso in considerazione viene utilizzata la seguente struttura:

$$\text{defaultGeneration} = \begin{bmatrix} 3 & 3 & 2 & 3 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 3 & 1 & 2 & 2 \\ 3 & 0 & 3 & 0 & 2 \\ 3 & 0 & 0 & 0 & 2 \end{bmatrix}$$

In particolare si ha che 1 -> casa grande, 2 -> casa media e 3 -> casa piccola. Questa matrice permette di estrarre un insieme di regole; ad esempio nel caso preso in considerazione, risulta che una casa piccola (3) può trovarsi a sinistra, a destra e sotto un'altra casa piccola, ma non può trovarsi sopra una casa grande (1).

2. Inizializzazione delle celle. All'inizio dell'algoritmo, ciascuna cella della matrice può contenere un qualsiasi valore intero in $[0, 3]$; l'idea di base consiste nello scegliere un punto casuale della matrice ed assegnare un valore.
3. Propagazione dei vincoli. Partendo dalla cella di cui è effettuata la scelta del valore, è necessario aggiornare tutte le celle circostanti sfruttando le regole calcolate nel passo (1) e propagando i vincoli lungo la matrice. Per fare questa operazione viene solitamente utilizzata una coda in cui vengono inserite tutte le celle i cui valori possibili sono stati alterati.

4. Entropia e collasso. Il procedimento viene ripetuto scegliendo la cella con l'entropia più bassa (ossia con il numero di possibilità più basso) e selezionando una possibilità tra quelle presenti nella cella.

Sfruttando questo procedimento è possibile generare villaggi differenti per struttura e dimensioni. Infatti la dimensione della mappa utilizzata per estrarre le regole non è vincolante per la generazione finale.

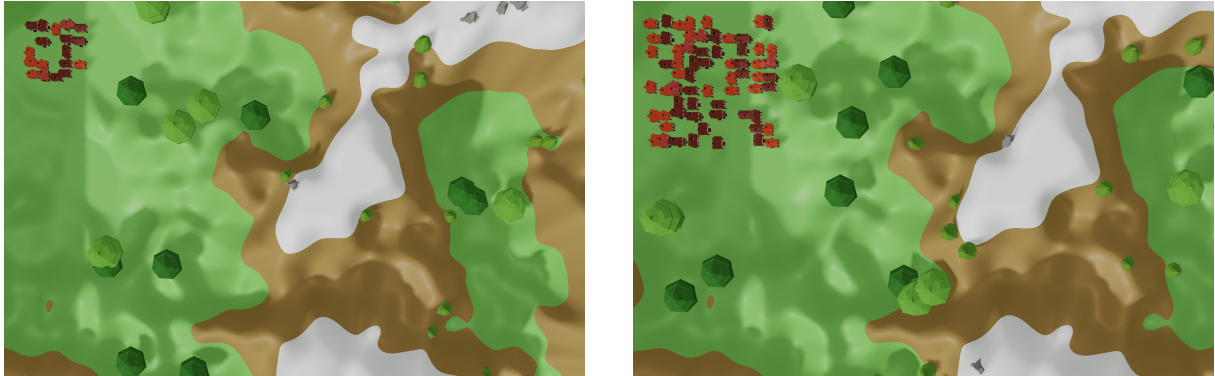


Figura 6: Alcuni esempi di generazione di villaggi con dimensione differente (5×5 e 10×10).

5 Conclusioni

Il progetto realizzato ha permesso di comprendere alcune delle principali tecniche utilizzate per la generazione procedurale di terreni in ambito gaming. In particolare è stata effettuata l'implementazione e la dimostrazione di tre tecniche algoritmiche fondamentali: l'improved Perlin noise [5], il Poisson disk sampling [1] e la wave function collapse [2]. Analizzando in dettaglio la pipeline implementata, il primo algoritmo è stato utilizzato per manipolare la struttura del terreno, mentre il secondo ed il terzo algoritmo sono stati applicati con l'obiettivo di distribuire gli assets sul terreno seguendo un criterio. La variazione di un algoritmo con un altro della stessa tipologia produce un risultato differente; per questa ragione è possibile considerare una pipeline di generazione procedurale come una struttura modulare, dove ogni modulo ha un compito ben preciso e contribuisce ad un aspetto della generazione. L'utilizzo di strutture dati differenti consente di gestire ciascun aspetto della generazione in modo atomico e contribuisce al mantenimento ed all'ampliamento del codice; ciò contribuisce ulteriormente alla massimizzazione della modularità del procedimento.

6 Risultato finale ed alcuni esempi di generazione

All'interno di questa sezione (Figura 7) sono esposti alcuni risultati dell'applicazione della pipeline di generazione del terreno. Ciascun piano è formato da 256×256 vertici e sono state selezionate 6 ottave per il Perlin noise; nelle prime due immagini gli assets sono distribuiti casualmente, mentre nelle ultime due sono raccolti in foreste.

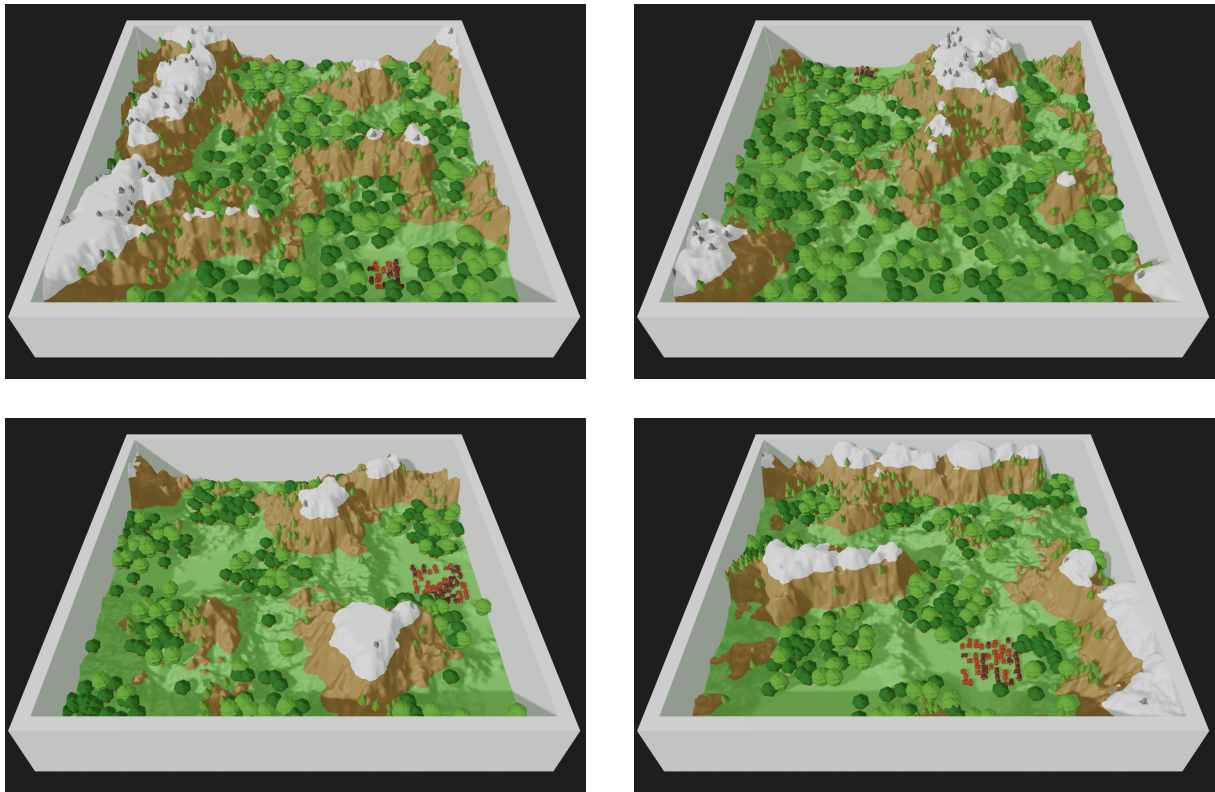


Figura 7: Alcuni esempi di generazione di terreni tramite la pipeline indicata in precedenza. La variazione pseudocasuale è riproducibile e controllata da un seed.

Riferimenti bibliografici

- [1] Robert Bridson. «Fast Poisson disk sampling in arbitrary dimensions.» In: *SIGGRAPH sketches* 10.1 (2007), p. 1.
- [2] Maxim Gumin. *WaveFunctionCollapse*. <https://github.com/mxgmn/WaveFunctionCollapse>. Accessed: 2026-07-29. 2016.
- [3] Benoit B Mandelbrot e John W Van Ness. «Fractional Brownian motions, fractional noises and applications». In: *SIAM review* 10.4 (1968), pp. 422–437.
- [4] Ken Perlin. «An image synthesizer». In: *ACM Siggraph Computer Graphics* 19.3 (1985), pp. 287–296.
- [5] Ken Perlin. «Improving noise». In: *Proceedings of the 29th annual conference on Computer graphics and interactive techniques*. 2002, pp. 681–682.